Files

main
Go to file

- assets
- .gitignore
- Makefile
- README.md
- commit.c
- commit.h
- file1.txt
- file2.txt
- hello.txt
- index.c
- index.h
- object.c
- oldREADME.md
- pes.c
- pes.h
- test_objects.c
- test_sequence.sh
- test_tree.c
- tree.c
- tree.h

PES-VCS

A minimal version control system implemented in C to understand how Git works internally.

This project implements:

- Content-addressable object storage (blob, tree, commit)
- A staging index
- Commit creation and history traversal
- End-to-end integration workflow

It also includes design answers for advanced topics:

- Q5.1 checkout
- Q5.2 dirty working directory detection
- Q5.3 detached HEAD
- Q6.1 garbage collection algorithm
- Q6.2 GC race condition

1. Build and Run

Prerequisites

- gcc
- OpenSSL development headers and library (`libssl-dev`)

Install (Ubuntu):

```
sudo apt update
sudo apt install -y build-essential gcc libssl-dev
```

Build Commands

```
make pes            # Build main binary
make test_objects   # Build Phase 1 test
make test_tree      # Build Phase 2 test
make test           # Run unit + integration tests
make clean          # Clean artifacts and .pes
```

CLI Commands

```
./pes init
./pes add <file>...
./pes status
./pes commit -m "message"
./pes log
```

2. Repository Layout

- `object.c` : Blob/tree/commit object read-write and integrity verification
- `tree.c` : Tree parse/serialize and tree construction from index
- `index.c` : Staging area load/save/add/status
- `commit.c` : Commit creation, HEAD/ref update, log walk
- `pes.c` : CLI command dispatch
- `test_objects.c` : Phase 1 unit tests

- `test_tree.c` : Phase 2 unit tests
- `test_sequence.sh` : Integration test sequence
- `assets/` : Evidence screenshots

3. Data Formats

Object Format

All objects are stored as:

```
"<type> <size>\0<data>"
```

Hashing rule:

- SHA-256 is computed over the full byte stream (header + data)
- Object path is derived from hash:
  - directory: first 2 hex chars
  - filename: remaining 62 hex chars

Tree Entry Format

Each entry is serialized as:

```
"<mode-octal> <name>\0<32-byte-binary-hash>"
```

Index File Format

One text entry per line in `.pes/index` :

```
<mode-octal> <64-char-hash> <mtime-seconds> <size> <path>
```

Commit Object Format

```
tree <tree-hash>
parent <parent-hash>          # omitted for first commit
author <name> <timestamp>
committer <name> <timestamp>

<message>
```

4. Phase-wise Implementation and Evidence

Phase 1: Object Storage

Implemented:

- Object write with atomic temp-file + rename
- Deduplication using content hash
- Object read with integrity verification

Evidence:

1. Phase 1 implementation commits and push history:

```
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ git add .
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ git commit -m "Phase 1: Implement object_write - header + SHA256 + atomic write"
[main 095ab69] Phase 1: Implement object_write - header + SHA256 + atomic write
 1 file changed, 48 insertions(+), 4 deletions(-)
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.07 KiB | 1.07 MiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/maddox095/PES1UG24CS539-pes-vcs
   1f34bb4..095ab69  main -> main
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ git commit -m "Phase 1: Implement object_read - parse header + integrity verify"
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   object.c

no changes added to commit (use "git add" and/or "git commit -a")
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ git add .
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ git commit -m "Phase 1: Implement object_read - parse header + integrity verify"
[main f1c685c] Phase 1: Implement object_read - parse header + integrity verify
 1 file changed, 45 insertions(+), 4 deletions(-)
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1012 bytes | 1012.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/maddox095/PES1UG24CS539-pes-vcs
   095ab69..f1c685c  main -> main
```

2. Phase 1 unit tests passing:

```
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$  make test_objects && ./test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$
```

3. Sharded object store paths under `.pes/objects`:

```
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$
```

4. Raw object bytes (`xxd`) showing header + content:

```
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$  make test_objects && ./test_objects && find .pes/objects -type
-1
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$  xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5a
471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$
```

Phase 2: Tree Objects

Implemented:

- Deterministic tree serialization (sorted entries)
- Safe tree parsing
- Tree object generation from staged index entries (including nested paths)

Evidence:

```
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$  make test_tree && ./test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$
```

Phase 3: Index (Staging Area)

Implemented:

- `index_load` : parse index file into memory
- `index_save` : atomic save with sorted entries
- `index_add` : stage file content as blob and update metadata
- `index_status` : staged/unstaged/untracked reporting

Evidence:

1. `init` , `add` , `status` workflow and resolved staging flow:

```
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$  ./pes init
Initialized empty PES repository in .pes/
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$  printf 'hello\n' > file1.txt && printf 'world\n' > file2.txt && ./pes a
d file1.txt file2.txt
Segmentation fault (core dumped)
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$  make pes && ./pes init && printf 'hello\n' > file1.txt && printf 'world
n' > file2.txt && ./pes add file1.txt file2.txt && ./pes status && cat .pes/index
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_sequence.sh
  untracked:  index.h
  untracked:  object.c
  untracked:  Makefile
  untracked:  pes.c
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  test_objects
  untracked:  test_tree.c
  untracked:  test_tree
  untracked:  commit.h
  untracked:  tree.h
  untracked:  index.c
  untracked:  pes.h
  untracked:  .gitignore
  untracked:  tree.c
  untracked:  README.md

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776680705 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776680705 6 file2.txt
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$
```

2. Stored `.pes/index` entries:

```
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776680705 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776680705 6 file2.txt
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$
```

Phase 4: Commit and Log

Implemented:

- Build root tree from index
- Create commit object with optional parent
- Update branch reference through HEAD
- Walk commit history with `pes log`

Evidence:

1. Clean rebuild, first commit, and log output:

```
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ make clean && make pes && make test_objects && make test_tree &&
.pes && ./pes init && printf 'Hello\n' > hello.txt && ./pes add hello.txt && ./pes commit -m "Initial commit" && ./pes l
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Initialized empty PES repository in .pes/
Committed: 888571528bdd... Initial commit
commit 888571528bddbd207627fcdbedbf9ebff69420f0fa19c47c7b91efe98defb0a6
Author: PES User <pes@localhost>
Date:   1776681334

    Initial commit
```

2. `.pes` internal files after commit:

```
  307  }
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/88/8571528bddbd207627fcdbedbf9ebff69420f0fa19c47c7b91efe98defb0a6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/refs/heads/main
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$
```

3. HEAD and branch reference correctness:

```
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ cat .pes/refs/heads/main
888571528bddbd207627fcdbedbf9ebff69420f0fa19c47c7b91efe98defb0a6
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$
```

Integration Test Evidence

End-to-end test (`make test-integration`) output:

```
maddox@deviant-ubuntu:~/Desktop/PES1UG24CS539-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 6c127667093e... Initial commit

Log after first commit:
commit 6c127667093ecb1e141ab892e09aad56bcd03f987ccfe1868580d3daafa28ec4
Author: PES User <pes@localhost>
Date:   1776681619

    Initial commit


--- Second Commit ---
Committed: dd1c9a86c895... Update file.txt

--- Third Commit ---
Committed: d477a4b61be3... Add farewell

--- Full History ---
commit d477a4b61be37eb0dd5a78540bbf915bdedb3099d9128536d44f23d287d8fece
Author: PES User <pes@localhost>
Date:   1776681619

    Add farewell

commit dd1c9a86c8952b2ee0ef011b9ae117c58e673f673a344c234266bca14653f4b0
Author: PES User <pes@localhost>
Date:   1776681619

    Update file.txt

commit 6c127667093ecb1e141ab892e09aad56bcd03f987ccfe1868580d3daafa28ec4
Author: PES User <pes@localhost>
Date:   1776681619

    Initial committial commiQ


--- Reference Chain ---
HEAD:
```

5. Q5 Answers

Q5.1 - `pes checkout <branch>`

Goal:

- Switch branch pointer in `.pes/HEAD`
- Materialize target commit's tree into working directory
- Delete files present in old tree but absent in new tree
- Avoid partial switch state

Recommended safe algorithm:

1. Acquire checkout lock (for example `.pes/checkout.lock`).
2. Resolve current HEAD branch and current commit/tree.
3. Resolve target branch, target commit, and target tree.
4. Run dirty check (Q5.2). If dirty, abort checkout.
5. Build two maps:
   - old snapshot: `path -> blob-hash`
   - new snapshot: `path -> blob-hash`
6. For each file in new snapshot:
   - read blob content
   - write to `path.tmp`
   - fsync temp file
   - rename `path.tmp` to final path

7. For each file in old snapshot that is not in new snapshot:
    - delete from working directory
8. Atomically update `.pes/HEAD` to `ref: refs/heads/<branch>`.
9. Release lock.

Failure handling:

- If any file write fails, stop and keep HEAD unchanged.
- This ensures no branch pointer points to a half-updated working tree.

Q5.2 - Dirty Working Directory Detection

Goal:

- Refuse checkout when tracked files have unstaged content changes.

Algorithm:

1. For every entry in index:
    - read filesystem metadata using `stat`
2. If both `mtime` and `size` match index:
    - treat as clean, continue
3. If metadata differs:
    - read file content
    - compute SHA-256 blob hash using object header format
    - compare against hash stored in index
4. If any mismatch is found:
    - mark working directory dirty
    - reject checkout

Why this is efficient:

- Fast path uses metadata only
- Expensive hashing runs only on candidates whose metadata changed

Q5.3 - Detached HEAD

Definition:

- HEAD contains a raw commit hash instead of `ref: refs/heads/<branch>`.

Behavior:

- New commits are still created
- No branch pointer is advanced
- Those commits can become hard to reach later

Recovery:

```
git branch <name> <detached-commit-hash>
```

Equivalent PES concept:

- Create `.pes/refs/heads/<name>` containing the detached commit hash
- Point `.pes/HEAD` to `ref: refs/heads/<name>`

6. Q6 Answers

Q6.1 - Garbage Collection Algorithm

Objective:

- Remove unreachable objects from `.pes/objects`

Mark-and-sweep design:

1. Enumerate all branch refs in `.pes/refs/heads/*`.
2. Initialize DFS/BFS stack with referenced commit hashes.
3. While stack not empty:
    - pop object hash
    - if already visited, continue
    - mark as reachable
    - parse object by type:
        - commit: push tree hash and parent hash (if present)
        - tree: push child blob/tree hashes
        - blob: no outgoing edges

4. Sweep phase:
    - iterate all object files in `.pes/objects`
    - delete any object not in reachable set

Complexity:

- Mark phase: `O(V + E)` over object graph
- Sweep phase: `O(N)` over on-disk objects
- With 100,000 commits and about 10 objects each, roughly 1,000,000 objects may be visited

Practical implementation notes:

- Use iterative traversal to avoid recursion depth issues
- Keep reachable hashes in a hash set for `O(1)` lookup

Q6.2 - GC Race Condition

Race scenario:

1. Thread A (commit) writes a new blob.
2. Thread B (GC) scans refs before commit object is written.
3. GC sees blob as unreachable and deletes it.
4. Thread A writes commit/tree referencing missing blob.
5. Repository becomes inconsistent.

How Git avoids this:

- Lock files for operations that mutate references or object namespace
- Grace period policy (recent objects are not collected)

Recommended PES safeguards:

1. Acquire repository-wide GC lock before sweep.
2. During GC, block commit/ref-update operations or use ref transaction locks.
3. Skip deleting objects newer than a grace window (for example 2 weeks).
4. Keep object creation atomic (`tmp` + `rename`) and ref updates atomic.

7. Current Scope and Future Work

Implemented in CLI today:

- `init` , `add` , `status` , `commit` , `log`

Design-only in this README (not yet exposed as CLI commands):

- `checkout`
- `gc`
- explicit detached-HEAD command flow

Future improvements:

- Recursive untracked file scanning
- Branch creation/listing/deletion commands
- Checkout with conflict handling
- Incremental and concurrent-safe garbage collection

8. Quick Validation Commands

```
make clean
make all
./test_objects
./test_tree
make test-integration
```